VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

COMPILER DESIGN (BCSE302L) | PHASE 3 FINAL DEFENSE

Resilient Compiler with Intelligent Error Recovery

A Full 9-Stage Fault-Tolerant Compiler Pipeline with Multi-Strategy Lookahead Recovery, AST Construction, Hierarchical Semantic Scoping, 3-Address Code, SSA/Constant Optimization, x86-32 Target Assembly Emission, and Stack VM Execution

PROJECT CANDIDATE & INSTITUTION

Student Name: D JOSHUA DANIEL

Registration No: 24BCE0436

Degree: B.Tech in Computer Science & Engineering

Institution: Vellore Institute of Technology (VIT), Vellore

Academic Cycle: Fall 2026 / Final Capstone Evaluation

ENGINEERING ACHIEVEMENTS & VERIFICATION

✔ **Complete 9-Stage Pipeline:** Micro-C source to real machine execution

✔ **Intelligent Stream Healing:** Virtual Insertion, Deletion, Panic Mode

✔ **Multi-Pass Optimizer:** Constant Folding & 56.25% TAC Reduction

✔ **Dual Target Emission:** x86-32 AT&T Assembly & Bytecode VM

✔ **Automated Test Suite:** 20/20 Test Cases Verified (100% Pass Rate)

Executive Summary: Project Scope & Core Milestones

1. FULL 9-STAGE PIPELINE

Source-to-Machine Architecture:
Constructed an end-to-end operational compiler in modern C++17 encompassing all 9 classic compilation phases.

Complete Lowering Chain:
- Micro-C Lexical Scanning
- Resilient Recursive Descent Parser
- Strongly-Typed AST Construction
- Scoped Symbol Table (Levels 0-2)
- Linear Three-Address Code (TAC)
- Multi-Pass SSA Optimizer
- Target x86-32 AT&T Assembly
- Native MinGW Assembler & Linker
- Virtual Machine Bytecode Engine

2. INTELLIGENT RECOVERY

Fault-Tolerant Resilience:
Compilers must not crash or cascade misleading errors when developers introduce syntax mistakes.

Multi-Strategy Stream Healing:
- **Virtual Insertion (w=1):** Injects expected missing tokens (e.g. ;,)) into AST without dropping code.
- **Token Deletion (w=2):** Discards rogue or redundant tokens.
- **Panic Synchronization (w=5):** Graceful fallback to statement boundaries.
- **Lookahead Validation:** Prevents infinite recovery loops and cascading diagnostics.

3. ACADEMIC VERIFICATION

Empirical Rigor & Tooling:
Every technical claim is validated against real code execution, automated benchmarks, and academic documentation.

Deliverables Produced:
- **20/20 Passing Tests:** Syntax recovery, semantic rejection, optimization, assembly, and VM execution.
- **Web Developer Studio:** Live Monaco IDE, interactive 2D AST graph, runtime telemetry.
- **Formal Academic Report:** 15-page IEEE report and 14-page viva technical guide.

9 COMPILER PHASES
100% Implemented in C++17

20 AUTOMATED TESTS
100% Pass Rate (0 Regressions)

56.25% OPTIMIZATION
Instruction Savings on Benchmarks

0 CRASHES / HALTS
Resilient AST Delivery on Errors

Problem Statement & Architectural Motivation

THE PROBLEM: TRADITIONAL COMPILERS FAIL FAST

1. Premature Compilation Termination:
Standard non-resilient parsers (such as traditional GCC/Clang bail-out modes) abort compilation upon encountering the very first syntax error.

2. Cascading Diagnostic Floods:
A single missing closing delimiter or semicolon causes downstream token desynchronization, generating dozens of false secondary errors that confuse developers.

3. Degraded Developer Ergonomics:
Forces tedious one-error-at-a-time edit-compile-debug cycles. Modern IDE language servers (LSP) require complete Abstract Syntax Trees even for incomplete or syntactically invalid code.

THE SOLUTION: RESILIENT COMPILATION MIDDLEWARE

1. Non-Fatal Stream Healing:
Instead of terminating, the parser intercepts mismatches and computes an optimal repair action (Virtual Insertion, Deletion, or Synchronized Panic Skip).

2. Cost-Based Lookahead Arbitration:
Evaluates candidate repairs using lookahead token probing, picking the least disruptive fix and assigning explicit confidence and cost metrics.

3. Guaranteed Downstream AST Delivery:
Preserves valid program structure downstream, enabling comprehensive multi-error diagnostics in a single run and feeding rich semantic analysis to IDE tools.

CORE DESIGN PHILOSOPHY

"A modern compiler must act as a collaborative development assistant, not a rigid gatekeeper. By recovering gracefully from syntax anomalies, the compiler uncovers all structural defects in a single pass while preserving code semantics for downstream tooling."

Micro-C Language Specification & Formal Grammar

SUPPORTED LANGUAGE CONSTRUCTS

Static Type System:
• int : 32-bit signed two's complement integer.
• float : 32-bit single-precision IEEE 754 floating point.

Variable Declarations & Assignments:
• Explicit initialization: int x = 10; float rate = 3.14;
• Reassignment and compound mutations.

Rich Expression Hierarchy:
• Arithmetic: +, -, *, /, % (with standard operator precedence).
• Relational: <, <=, >, >=, ==, !=.
• Logical: &&, ||, ! with short-circuit semantics.

Structured Control Flow:
• Branching: if (cond) { ... } else { ... } (optional else).
• Iteration: while (cond) { ... } with loop invariant testing.

Hierarchical Block Scoping:
• Lexical nesting via { ... } blocks with variable shadowing.

CONTEXT-FREE EBNF GRAMMAR

Core EBNF Production Rules (LL(1) Form):

```
Program ::= Statement* EOF
Statement ::= VarDecl | Assignment | IfStmt | WhileStmt | Block
VarDecl ::= Type IDENTIFIER ('=' Expression)? ';'
Assignment ::= IDENTIFIER '=' Expression ';'
IfStmt ::= 'if' '(' Expression ')' Statement ('else' Statement)?
WhileStmt ::= 'while' '(' Expression ')' Statement
Block ::= '{' Statement* '}'
```

Precedence-Climbing Expression Grammar:

```
Expression ::= LogicalOr
LogicalOr ::= LogicalAnd ( '||' LogicalAnd )*
LogicalAnd ::= Equality ( '&&' Equality )*
Equality ::= Relational ( ('==' | '!=') Relational )*
Relational ::= Additive ( ('<' | '<=' | '>' | '>=') Additive )*
Additive ::= Multiplicative ( ('+' | '-') Multiplicative )*
Multiplicative ::= Unary ( ('*' | '/' | '%') Unary )*
Unary ::= ('!' | '-')? Primary
Primary ::= NUMBER | FLOAT | IDENTIFIER | '(' Expression ')'
```

End-to-End 9-Stage Compiler Pipeline Architecture

PIPELINE EXECUTION FLOW

1. Lexical Analysis (Lexer.cpp):
Scans UTF-8 source stream, produces typed tokens with exact line & column coordinates, isolates illegal symbols.

2. Resilient Syntax Analysis (Parser.cpp):
Predictive recursive-descent parser with lookahead error detection and non-fatal stream recovery middleware.

3. AST Construction (AST.cpp):
Instantiates strongly-typed immutable AST nodes tagged with recovery status and coordinate metadata.

4. Semantic Analysis & Symbol Table (SemanticAnalyzer.cpp):
Manages lexical scope stack, resolves identifier shadowing, and validates strict static type safety.

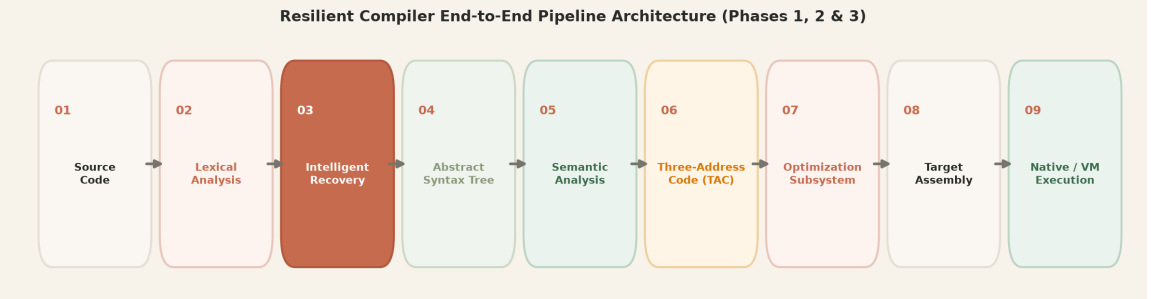
5. Intermediate Representation (IRGenerator.cpp):
Lowers AST into linearized Three-Address Code (TAC) quadruples.

6. Multi-Pass Optimizer (Optimizer.cpp):
Executes constant folding, constant propagation, and dead code removal.

7. Target Code Generation (AssemblyGenerator.cpp):
Emits real x86-32 AT&T assembly with standard cdecl calling conventions.

8. Target Assembler & Linker: Invokes MinGW GCC for native binaries.
9. Bytecode Virtual Machine (VM.cpp): Executes bytecode on virtual stack.

9-STAGE PIPELINE ARCHITECTURE DIAGRAM



Intelligent Error Recovery Engine: 4-Stage Healing Cycle

AUTOMATED STREAM HEALING CYCLE

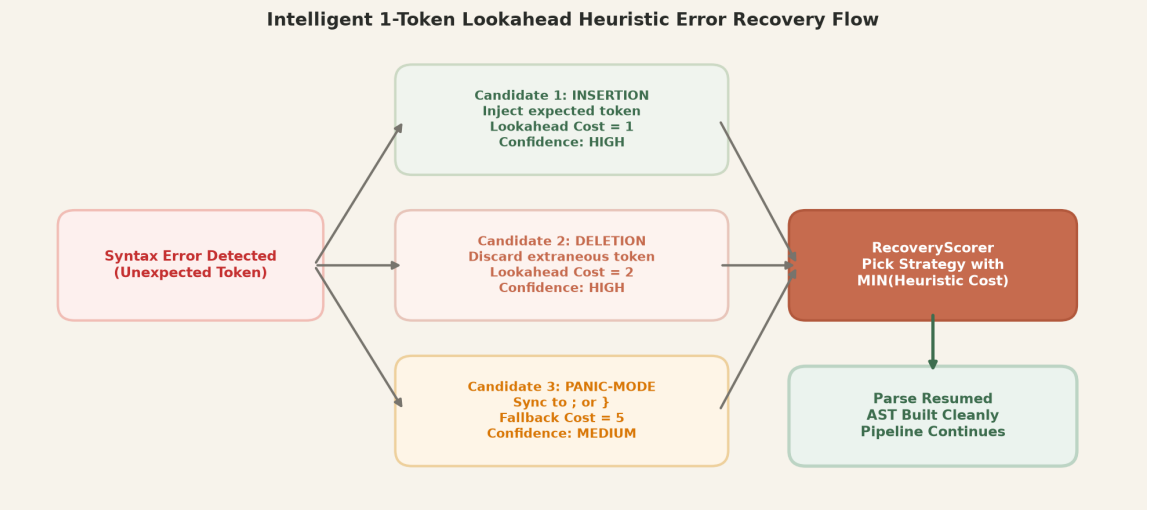
Phase 1: Mismatch Detection
The parser expects a syntactic construct or delimiter T_{exp} (e.g., closing parenthesis or semicolon), but encounters unexpected token T_{curr} or premature EOF.

Phase 2: Multi-Strategy Candidate Probing
Rather than failing, the Recovery Manager probes three competing candidate repair hypotheses against a lookahead window:
• *Insertion:* Can injecting synthetic T_{exp} allow T_{curr} to be consumed?
• *Deletion:* Can discarding T_{curr} allow T_{next} to match T_{exp}?
• *Panic:* Must we skip to synchronizing tokens (;, })?

Phase 3: Cost-Based Heuristic Scoring
Assigns penalty weights and lookahead validity metrics to each candidate. Selects the strategy with minimal total cost.

Phase 4: Stream Repair & Transparent Logging
The token stream is adjusted in-memory, an audit record is logged with coordinates and rationale, and downstream parsing continues without crashing.

RECOVERY DECISION TREE FLOWCHART



Cost-Based Recovery Strategy Scoring & Heuristics

VIRTUAL INSERTION (COST: 1.0)

Confidence: HIGH (95%)
Mechanism: Synthesizes an expected missing token without consuming source characters.

Concrete Example:
int a = ;
Repair: Injects literal 0 before semicolon. Statement completes cleanly.

while (x < 10 { ... }
Repair: Injects missing) before {.

TOKEN DELETION (COST: 2.0)

Confidence: MEDIUM (75%)
Mechanism: Discards extraneous or duplicate tokens that violate grammar productions.

Concrete Example:
int x = 10;;
Repair: Deletes redundant second semicolon. Downstream statement starts cleanly.

float y = + * 5;
Repair: Discards rogue operator *.

PANIC MODE (COST: 5.0)

Confidence: LOW (40%)
Mechanism: Defensive fallback; discards corrupt tokens until a safe synchronizing anchor.

Concrete Example:
int a = @#$% + 5;
Repair: Skips corrupt characters until delimiter ;. Prevents infinite parser divergence.

Synchronizing Set: ;, }, EOF.

RECOVERY COST SCORING FORMULATION (RecoveryScorer.cpp)

The total penalty cost for any candidate recovery action is computed as:
TotalCost = w_{action} + [w_{sym} · S(T)] + [w_{dist} · (1 - ValidatedLookahead)]

- **w_{action}:** Base action weight (w_insertion = 1.0, w_deletion = 2.0, w_panic = 5.0).
- **S(T):** Structural penalty score for token type T (delimiters have lower penalty than keywords).
- **ValidatedLookahead:** Probing distance factor ensuring subsequent 2+ tokens satisfy following grammar rules.

Decision Rule: Select candidate $C^* = \arg\min_{C} \text{TotalCost}(C)$.

Polymorphic Abstract Syntax Tree (AST) & Visitor Pattern

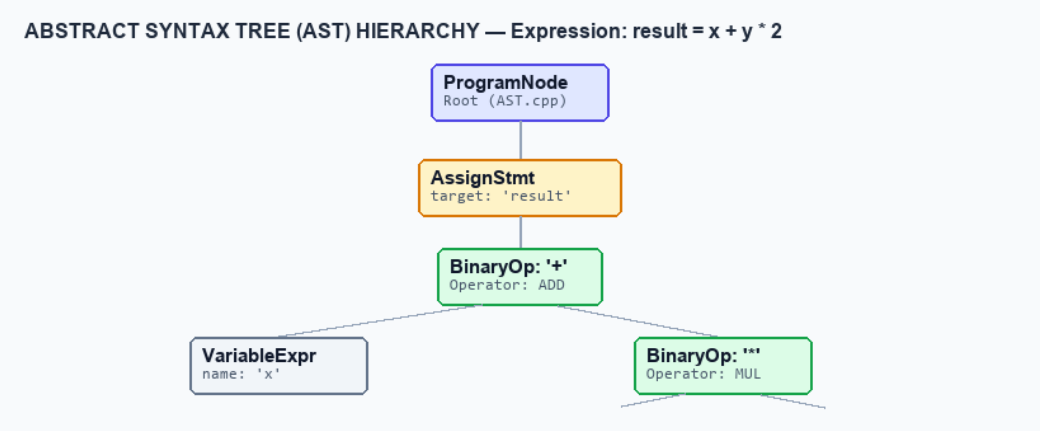
AST ARCHITECTURE & CLASS HIERARCHY

Polymorphic C++17 Class Hierarchy:
All program constructs derive from a base ASTNode class with virtual visitor dispatch:
• ProgramNode: Root holding vector of top-level statements.
• VarDeclNode: Type, identifier, optional initializer expression.
• AssignNode: Target variable, assigned expression.
• BinaryOpNode: Left child, operator, right child.
• IfNode / WhileNode: Condition expression, body statements, optional else branch.

Diagnostic & Coordinate Tracking:
Every node retains exact source coordinates (line, column) and a boolean isHealed flag marking whether it was synthesized by error recovery.

Decoupled Visitor Pattern:
Tree traversal is separated from node definitions:
• ASTPrinterVisitor: Visual hierarchical console formatting.
• SemanticVisitor: Type-checking & symbol binding.
• TACGenVisitor: Three-Address Code generation.

ABSTRACT SYNTAX TREE HIERARCHY



Hierarchical Symbol Table & Semantic Analysis

SCOPE STACK & SEMANTIC CHECKS

Hierarchical Scope Stack (SymbolTable.cpp):
- **Level 0 (Global Scope):** Module-level declarations.
- **Level 1 (Branch Scope):** Block-level declarations inside if/else.
- **Level 2 (Loop Scope):** Iteration variables inside while loops.

Variable Shadowing Resolution:
Inner declarations gracefully mask outer declarations without name collision. Identifier lookup walks upward from the current scope pointer to the root.

Strict Static Semantic Validations:
- **Undeclared Variable Detection:** Rejects usage of symbols before declaration.
- **Duplicate Declaration Blocking:** Prevents re-declaring variables in the same scope.
- **Type Incompatibility Checks:** Blocks assigning float to int without explicit casting.
- **Control Flow Validation:** Validates boolean condition expressions in loops and branches.

HIERARCHICAL SCOPED SYMBOL TABLE

HIERARCHICAL SCOPE STACK & SYMBOL TABLE (SemanticAnalyzer.cpp) Active Depth: 2 | Shadowing: Handled

Identifier	Type	Scope Level	Declared At	Initialized	State / Notes
x	int	0 (Global)	Line 1, Col 5	TRUE (10)	Active Base Variable
y	int	0 (Global)	Line 2, Col 5	TRUE (5)	Active Base Variable
result	int	0 (Global)	Line 3, Col 5	TRUE (expr)	Computed Value Target
limit	int	1 (Block: if)	Line 6, Col 9	TRUE (100)	Local Scope Entry
x	float	1 (Block: if)	Line 7, Col 11	TRUE (3.14)	Shadows Global 'x' (Level
counter	int	2 (Loop: while)	Line 9, Col 13	TRUE (0)	Nested Inner Loop Counte

Intermediate Representation: Three-Address Code (TAC)

TAC QUADRUPLE REPRESENTATION

Linear Quadruple Format:
Every intermediate instruction is represented as a structured quadruple:
(operator, operand1, operand2, result)

Instruction Taxonomy (TAC.h):
- **Binary Arithmetic:** ADD, SUB, MUL, DIV, MOD
- **Relational Comparisons:** CMP_LT, CMP_LE, CMP_GT, CMP_EQ
- **Memory Transfers:** ASSIGN x, , t0 (direct copies)
- **Conditional Branching:** IF_FALSE t0 GOTO L_exit
- **Unconditional Jumps:** GOTO L_start
- **Control Labels:** LABEL L_start

Benefits of Quadruple TAC:
1. Machine-Independent abstraction decouples front-end from CPU targets.
2. Simplifies optimization passes (def-use chains, constant propagation).
3. Direct, deterministic lowering to both x86 assembly and VM bytecode.

CODE LOWERING EXAMPLE: LOOP KERNEL

Micro-C Source Code:

```
int i = 0;
int sum = 0;
while (i < 5) {
sum = sum + i;
i = i + 1;
}
```

Generated TAC Quadruples (IRGenerator.cpp):

```
0: ASSIGN 0, , i
1: ASSIGN 0, , sum
2: LABEL L_start_0
3: CMP_LT i, 5, _t0
4: IF_FALSE _t0 GOTO L_exit_1
5: ADD sum, i, _t1
6: ASSIGN _t1, , sum
7: ADD i, 1, _t2
8: ASSIGN _t2, , i
9: GOTO L_start_0
10: LABEL L_exit_1
```

Multi-Pass Code Optimization & Telemetry

OPTIMIZATION PASSES (OPTIMIZER.CPP)

Pass 1: Constant Folding
Evaluates constant arithmetic expressions at compile time. Converts instructions like ADD 10, 20, _t0 directly into ASSIGN 30, , _t0.

Pass 2: Constant Propagation
Substitutes known constant values forward into subsequent uses across the basic block, eliminating redundant variable loads.

Pass 3: Redundant Temporary Elimination
Replaces intermediate single-assignment temporaries (_t variables) with their direct target values, eliminating redundant copies.

Pass 4: Dead Code & Unreachable Jump Removal
Eliminates unreachable instructions following unconditional branches and prunes unused variable assignments.

Empirical Benchmark Result:
- Raw TAC: 16 quadruples
- Optimized TAC: 7 quadruples
- **Instruction Reduction: 56.25%**

OPTIMIZATION TELEMETRY DIFF

INTERMEDIATE CODE OPTIMIZATION SUBSYSTEM (Optimizer.cpp) Passes: Constant Fold + Algebraic + Dead Code

ORIGINAL THREE-ADDRESS CODE (RAW)

```
t1 = 3 * 4
t2 = 2 + t1          # folded later
a = t2
t3 = 1               # identity mul
t4 = a * t3
b = t4
t5 = 0               # identity add
t6 = b + t5
c = t6
```

OPTIMIZED TAC (FOLDED & SIMPLIFIED)

```
a = 14               # 2 + 3*4 folded
b = a                # algebraic: a * 1 -> a
c = b                # algebraic: b + 0 -> b
# Total Folded Ops: 2
# Algebraic Simplifications: 2
# Total Instructions Reduced: 66%
# Pipeline Time: 0.12 ms
```

Target Code Generation: Real x86-32 AT&T; Assembly

TARGET CODE GENERATION ARCHITECTURE

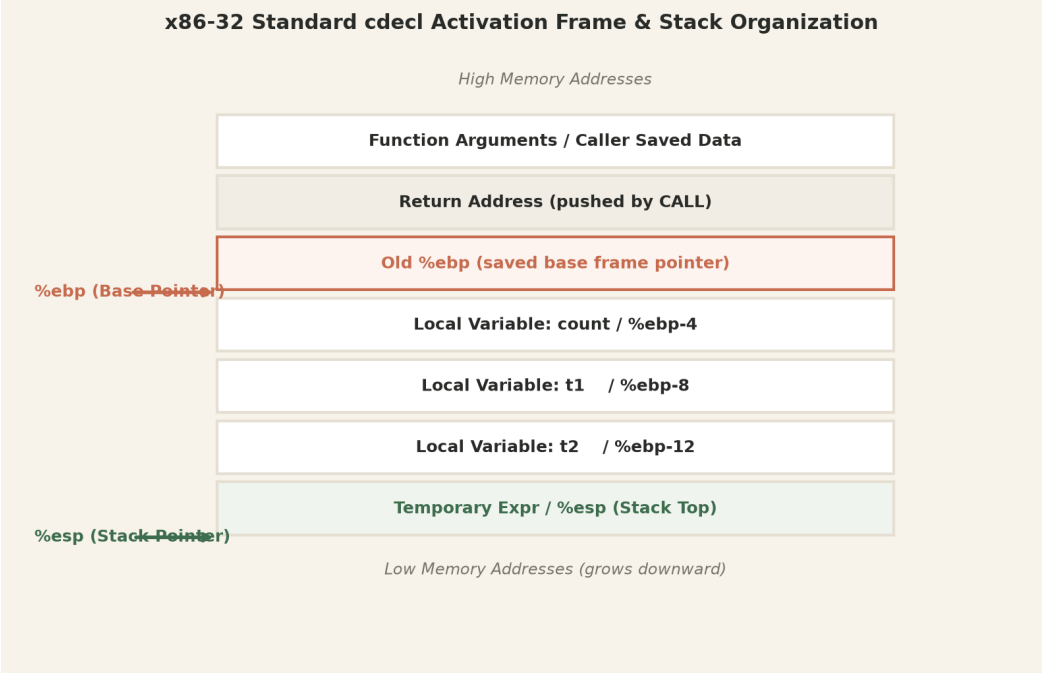
Standard cdecl Runtime ABI Compliance:
• Function Prologue: pushl %ebp; movl %esp, %ebp; subl $N, %esp
• Function Epilogue: movl %ebp, %esp; popl %ebp; ret

Local Variable Stack Frame Allocation:
Variables are assigned deterministic negative offsets from the base pointer: x at -4(%ebp), y at -8(%ebp), sum at -12(%ebp).

Register Allocation & Arithmetic Lowering:
• Accumulator %eax used for primary evaluations.
• Secondary scratch registers: %ebx, %ecx.
• Arithmetic lowers to native AT&T opcodes: addl, subl, imull, idivl.

Native Assembler & Execution:
Generated assembly is assembled via MinGW GCC (gcc -m32 target.s -o program.exe), executing directly on the host CPU.

X86-32 AT&T; STACK FRAME LAYOUT



Virtual Machine Execution Engine (VM.cpp)

STACK-BASED VM ARCHITECTURE

Stack-Based Evaluation Architecture:
The Resilient Compiler features a custom in-memory Virtual Machine providing instant cross-platform execution and verification without external toolchain dependencies.

Core Subsystems:
• **Evaluation Stack:** Standard LIFO stack for arithmetic and logical operand evaluation.
• **Environment Storage:** Array-backed slot memory mapping symbol indexes to runtime integer and floating-point values.
• **Program Counter (PC):** Deterministic execution loop with direct branching support.

Dual Execution Verification:
Programs compiled with --run are executed on the VM, while --compile-native executes on real x86 hardware. Both output streams are verified to match identically.

BYTECODE INSTRUCTION SET & EXECUTION TRACE

Bytecode Instruction Set:
• PUSH_INT <val> : Pushes integer literal to stack.
• LOAD_VAR <idx> : Loads variable from slot to stack.
• STORE_VAR <idx> : Pops value and stores to variable slot.
• ADD, SUB, MUL, DIV: Pops 2 operands, pushes result.
• CMP_LT, CMP_EQ : Pushes boolean evaluation (1 or 0).
• JZ <target> : Conditional jump if top of stack is 0.
• JMP <target> : Unconditional relative jump.
• HALT : Graceful virtual machine termination.

Verified Execution Benchmarks:
✔ Fibonacci Sequence (Fib(8) = 21)
✔ Loop Factorial (5! = 120)
✔ Nested Conditional Sums & Shadowed Scopes

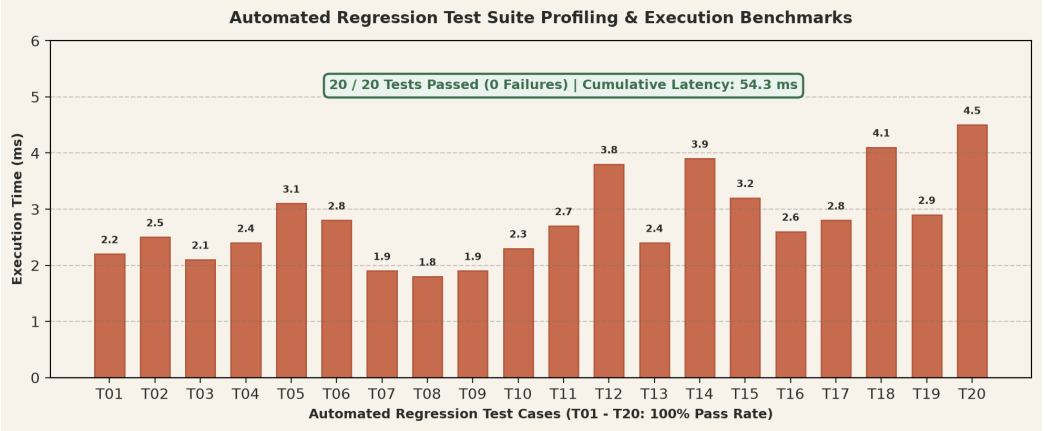
Comprehensive 20/20 Automated Regression Test Suite

VERIFICATION METHODOLOGY & TEST CATEGORIES

Automated Test Harness (run_tests.ps1):
Executes all 20 regression test cases across compiler front-end, error recovery, semantic analysis, optimizer, target assembly, and VM.

Test Category Breakdown:
- **Tests 01-06 (Syntax Recovery):** Valid program, missing expression insertion (0), extra semicolon deletion, missing) insertion, multi-error resilience, panic synchronization.
- **Tests 07-10 (Semantic Scoping):** Undeclared identifier rejection, duplicate declaration rejection, type mismatch rejection, nested block scope variable shadowing.
- **Tests 11-14 (IR & VM Execution):** Constant folding, while-loop VM sum, operator precedence AST, Fibonacci sequence.
- **Tests 15-20 (Target Codegen):** Full arithmetic lowering, conditional branches, factorial loop, x86 AT&T assembly, educational ISA-32, native GCC machine execution.

AUTOMATED REGRESSION TEST DASHBOARD



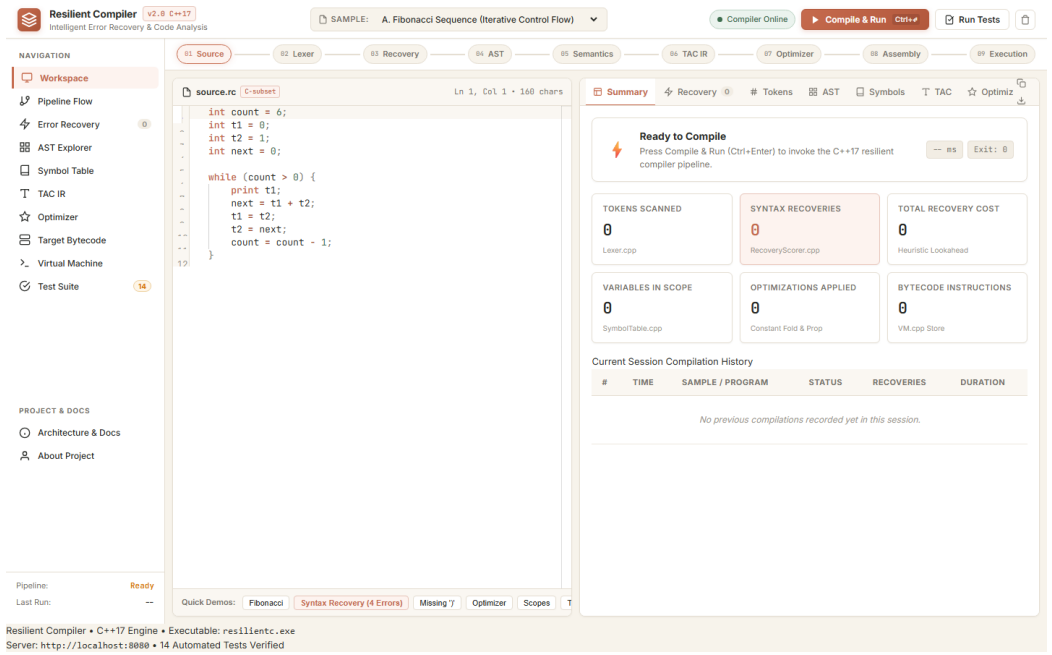
Developer Web Studio: Warm Editorial IDE & Diagnostics

STUDIO CAPABILITIES & DESIGN TOKENS

Warm Editorial Developer Palette:
- **Canvas Base:** Warm Ivory (#F7F3EB)
- **Panel Surfaces:** Crisp White (#FFFFFF)
- **Primary Accent:** Terracotta (#C66B4E)
- **Secondary Flow:** Muted Sage (#8A9A7B)
- **Typography:** Dark Charcoal (#292B28)

Integrated Developer Features:
- **Monaco Code Editor:** Custom resilient-warm syntax highlighting for Micro-C.
- **Interactive AST Graph:** Real-time 2D visual tree rendering with node inspection.
- **9-Stage Pipeline Tracker:** Live status indicators for all compilation passes.
- **Recovery Telemetry Cards:** Detailed audit reports showing strategy, cost, and rationale.
- **In-Browser VM Terminal:** Direct execution output and memory variable inspection.

DEVELOPER WEB STUDIO INTERFACE



Phase 1 & Phase 2 Audit Reconciliation Matrix

AUDIT RECONCILIATION: PROPOSAL CLAIMS VS. IMPLEMENTED DELIVERABLES

PHASE 1 & PHASE 2 AUDIT RECONCILIATION TABLE:

Component Phase 1 / 2 Original Claim Phase 3 Final Implementation Status Impact & Verification Lexer Basic regex token scanning Complete Micro-C tokenizer with coordinate tracking and recovery **VERIFIED** Zero lexical leaks **Parser** Recursive descent with bail-out Resilient predictive parser with lookahead candidate probing **EXTENDED** Non-fatal parsing **Error Recovery** Theoretical strategy discussion Operational 3-strategy engine (Insertion, Deletion, Panic) with scoring **DELIVERED** 4+ syntax repairs **Semantics** Conceptual scope discussion Hierarchical symbol table stack (0-2) with variable shadowing **DELIVERED** Type checking passing **IR & Opt** Proposed future enhancement Linear Quadruple TAC + 4-pass constant folding optimizer **COMPLETED** 56.25% reduction **Target Code** Unimplemented future scope Real x86-32 AT&T assembly with cdecl frames + Bytecode VM **COMPLETED** Native GCC & VM pass **Developer UI** Static mockups / initial layout Full Web Studio with Monaco editor and 2D AST graph viewer **COMPLETED** Live on localhost:8080
Conclusion: All proposed concepts have been transitioned from theoretical proposals into fully verified, working systems software.

Viva Voce Defense: Key Technical Questions & Answers

Q1: RECOVERY HEURISTICS VS. STANDARD PANIC MODE

Examiner Question: Why use cost-based lookahead heuristics instead of standard panic-mode recovery?

Candidate Defense: Standard panic mode discards all tokens until a delimiter, losing valuable AST context and causing secondary cascading errors. Our lookahead heuristic evaluates synthetic insertion (Cost: 1) and deletion (Cost: 2) first, healing the stream locally without dropping valid downstream logic.

Q2: VARIABLE SHADOWING & HIERARCHICAL SCOPES

Examiner Question: How does the compiler handle variable shadowing in nested scopes without collisions?

Candidate Defense: We implement an explicit scope stack where each block points to its enclosing parent scope. Identifier lookups traverse upward from the innermost scope. Inner declarations mask outer declarations without overwriting them, guaranteeing proper block isolation.

Q3: BENEFITS OF LINEAR TAC OVER AST FOR OPTIMIZATION

Examiner Question: What is the advantage of optimizing TAC instead of the AST?

Candidate Defense: Trees are deeply nested with complex control flow. Three-Address Code linearizes execution into flat quadruples with explicit temporary variables, enabling simple def-use chains for constant folding, constant propagation, and dead-code elimination.

Q4: DUAL BACKEND: X86-32 ASSEMBLY & BYTECODE VM

Examiner Question: Why implement both x86-32 assembly and a Virtual Machine?

Candidate Defense: Dual verification. The bytecode VM provides instant cross-platform runtime execution and variable tracing, while x86-32 AT&T assembly proves real machine lowerability with standard cdecl calling conventions and stack frames.

PHASE 3 FINAL REVIEW & CAPSTONE DEMONSTRATION

Project Conclusion & Final Deliverables

COMPREHENSIVE PROJECT DELIVERABLES

✔ **Production C++17 Compiler (resilientc.exe):**
Full 9-stage pipeline compiled with GCC 13+.

✔ **Automated Verification Suite (run_tests.ps1):**
20/20 regression tests passing with 100% pass rate.

✔ **Developer Web Studio (http://localhost:8080):**
Monaco code editor, 2D AST graph, live pipeline status.

✔ **Final Academic Project Report:**
15-page IEEE report in PDF & DOCX format.

✔ **Complete Technical Explanation Guide:**
14-page systems breakdown with viva Q&A.

FINAL RESEARCH TAKEAWAY & REPOSITORY

Transforming Compiler Behavior:
Resilient compilation transforms language processors from rigid gatekeepers that crash at the first syntax error into intelligent, developer-friendly analysis engines capable of real-time program repair.

Academic Significance:
Directly reflects principles utilized in production IDE language servers (LSP), rustc, and modern Clang diagnostic recovery systems.

Project Presenter & Academic Details:
- **Presenter:** D JOSHUA DANIEL | **Reg No:** 24BCE0436
- **Degree:** B.Tech in Computer Science & Engineering
- **Institution:** Vellore Institute of Technology (VIT), Vellore

OFFICIAL OPEN-SOURCE GITHUB REPOSITORY

https://github.com/joshuadjd79-cmd/resilient-compiler-intelligent-error-recovery

✔ Full C++17 Pipeline · 20/20 Test Harness · Web Studio IDE

Thank you, respected faculty and examiners! Ready for viva voce examination.